PEX 03
Autonomous First Aid Kit Delivery

CS 472 – Autonomous Systems Integration · Spring 2026

300 Points · Due: M40 at 10:30

"A downed pilot is stranded in an area where we only have an idea of their location. This individual is injured, must receive medical supplies, and then be extracted ASAP. We need to devise a way, using a drone and AI, to locate this person, deliver much-needed first aid, and then report their location for extraction."

Help Policy:	AUTHORIZED: Any resource, except another team's solution.
Team effort:	Never copy another team's work or submit it as your own.
Documentation:	Project log required — missing or vague = −12% penalty.

Contents

1 Background and Context **3**
1.1 The Real-World Problem . . . 3
1.2 Why Autonomy Matters . . . 3
1.3 The Target . . . 3

2 Project Description **4**

3 What You Will Learn **4**
3.1 Autonomous Systems Architecture . . . 4
3.2 Computer Vision in a Real Deployment . . . 4
3.3 Sensor Fusion and Geospatial Reasoning . . . 5
3.4 Drone Control via DroneKit and MAVLink . . . 5
3.5 Engineering Judgement and Quantitative Design . . . 5

4 Mission Objective **6**

5 The Mission State Machine **6**
5.1 State Diagram . . . 6
5.2 State Reference Table . . . 7
5.3 Governing Design Rules . . . 7

6 Why the States Are Designed This Way **7**

7 Challenges You Will Face **8**

8 Files Provided to You **10**

9 Provided Module API Reference **10**
9.1 object_tracking_y8_histo.py . . . 10
9.2 pex03_utils.py . . . 11
9.3 drone_lib.py . . . 11

10 Student TODO Reference **12**

11 Problem 1 — Detection Model (YOLOv8 VisDrone) **13**
11.1 Known Model Limitations . . . 13
11.2 Two-Phase Perception Pipeline . . . 14

12 Problem 2 — Target Centering (TARGET Mode) **14**
12.1 The Pixel Error Model . . . 14
12.2 Pixel-to-Meter Conversion (derive this for your altitude) . . . 14
12.3 Two-Speed Correction Strategy . . . 15

13 Problem 3 — Ground Distance Estimation **15**
13.1 Camera Geometry Approach (no rangefinder) . . . 15
13.2 Rangefinder Approach (if available) . . . 16

13.3 Computing and Flying to the Drop Point . . . 16

14 Problem 4 — Descent and Package Release (DELIVER Mode) 16
14.1 Two-Phase Descent Design . . . 17

15 Development Guide 17
15.1 Stage 1 — Camera and Detection (no drone) . . . 17
15.2 Stage 2 — Configuration and Startup . . . 18
15.3 Stage 3 — SITL Connectivity . . . 18
15.4 Stage 4 — Centering Loop (TARGET Mode) . . . 18
15.5 Stage 5 — CONFIRM Sequence . . . 18
15.6 Stage 6 — Delivery Geometry (Unit Test First) . . . 18
15.7 Stage 7 — Full Integration (Real Camera, SITL) . . . 18
15.8 Stage 8 — Live Flight . . . 18

16 SITL Testing Guide 18

17 Tunable Parameters 19

18 Hardware Reference 20

19 Documentation Requirements 20
19.1 Help Documentation . . . 20
19.2 Project Log . . . 20

20 Evaluation and Grading 21
20.1 Competitive Ranking . . . 21
20.2 Scoring Dimensions . . . 21
20.3 Teams That Fail to Execute . . . 21

A Configuration File Reference (pex03_config.json) 22

1. Background and Context

1.1 The Real-World Problem

Search and rescue operations in austere environments present one of the most difficult logistics challenges in emergency response. When a person is injured in a remote area — a downed pilot, a hiker with a broken leg, a soldier separated from their unit — the time between injury and first medical intervention is often the difference between survival and death. The *golden hour* is not a metaphor.

The fundamental problem is access. Terrain, hostile activity, weather, or simply distance may prevent first responders from reaching the casualty quickly enough. A ground team that takes 45 minutes to reach someone who needed a tourniquet in the first ten minutes has arrived too late regardless of their skill.

Unmanned aerial systems can fly directly to a GPS coordinate at speeds no ground vehicle can match, over terrain no vehicle can navigate, without putting another person at risk. A drone carrying a first-aid kit can reach a downed pilot in minutes and deliver critical supplies — a tourniquet, a pressure dressing, a space blanket — that buy time until a full rescue team arrives.

1.2 Why Autonomy Matters

An operator controlling a drone manually over a search area must simultaneously maintain visual or video contact, manage the flight, watch for obstacles, and identify the casualty. That cognitive load degrades performance rapidly and scales poorly as the search area grows. A system that can autonomously search, detect, confirm, and deliver — with a human only monitoring — multiplies the operator's effective reach dramatically and frees them to coordinate the broader rescue operation.

This is the operational context for PEX 03. You are not building a toy. You are building a system that could save a life if it works correctly, and waste critical time or cause harm if it does not. Every engineering decision you make — the confidence threshold, the delivery radius, the descent velocity — has a physical consequence. You will be expected to derive your choices from first principles and defend them.

1.3 The Target

A dressed mannequin standing in an open field represents the downed pilot. Your drone does not know the mannequin's location in advance. It must find the mannequin autonomously during its search flight, verify the sighting, maneuvere overhead, compute a drop point from sensor data, and lower the package gently enough that the contents survive.

Important

The care package contains eggs. This is intentional. Eggs break under the same forces that would render a medical kit useless. Controlling descent velocity, managing tether swing, and choosing a safe delivery radius are core engineering requirements — not incidental details.

2. Project Description

PEX 03 is the capstone programming exercise in CS 472: Autonomous Systems Integration. It asks you to apply every skill developed throughout the course and integrate them into a single working autonomous system.

The exercise integrates five areas simultaneously:

- **Computer vision.** A YOLOv8 neural network trained on aerial VisDrone imagery detects people from altitude. An HSV color-histogram identity tracker then maintains a per-frame lock — without relying on pixel position, making it robust to the camera motion caused by the drone's own correction commands.
- **Autonomous flight control.** DroneKit and MAVLink commands arm, navigate, and maneuvere the drone through every mission phase, including real-time body-frame velocity corrections during the centering loop.
- **Sensor fusion and geospatial mathematics.** The camera's field-of-view geometry, combined with the drone's GPS altitude and the IMU-measured camera tilt angle, are used to estimate the horizontal ground distance to the target *without a dedicated rangefinder*. That distance is projected to a GPS drop coordinate via bearing projection.
- **IMU integration.** The camera's built-in accelerometer and gyroscope measure the actual camera tilt angle before takeoff — replacing a manually configured constant with a measured value and improving distance estimation accuracy across all teams and hardware configurations.
- **State machine design.** A five-state machine governs all mission logic and enforces correct transition ordering. All transitions are visible in a single method, making the code readable and debuggable under real operational pressure.

Your primary deliverable is `pex03.py`, which contains the `DroneMission` class and its `conduct_mission()` event loop. A skeleton with detailed comments and `# TODO` markers is provided; you fill in the implementation.

3. What You Will Learn

3.1 Autonomous Systems Architecture

- How to design a state machine for a safety-critical system where an incorrect transition has immediate physical consequences.
- Why all state-transition decisions belong in a single method and what happens when they are scattered across helper functions.
- When a *blocking* subroutine is the correct architectural choice over a non-blocking event loop, and how to reason about what the drone is doing during a blocking call.
- How a clean separation between *what to do next* (state machine) and *how to do it* (helper methods) makes a complex system testable.

3.2 Computer Vision in a Real Deployment

- Why appearance-based identity tracking (HSV color histograms) is fundamentally more robust under camera ego-motion than position-based tracking (CSRT, Kalman filters) — and what

"ego-motion" means for a drone correction loop.

- How a two-phase detect-then-track pipeline works: heavyweight YOLO detection to acquire a lock, lightweight histogram matching to maintain it.
- The critical difference between a *non-committing* detection scan (`check_for_initial_target`) and *committing* the tracker to a target (`set_object_to_track`) — and why the drone must fly back to the sighting location before doing the latter.
- Real-world model limitations: confidence score distributions, orientation sensitivity, altitude-dependent scale, and the cost of false positives in flight time.

3.3 Sensor Fusion and Geospatial Reasoning

- How to combine camera geometry (field of view, tilt angle, pixel row of target) with GPS altitude to estimate horizontal ground distance — without any dedicated rangefinder hardware.
- Why IMU-measured tilt is more accurate than a manually configured constant, and how to obtain a stable measurement before the motors create vibration.
- How a blocking depth-averaging window reduces noise, and why drone stability during that window directly affects delivery accuracy.

3.4 Drone Control via DroneKit and MAVLink

- The difference between AUTO mode (autopilot executes its waypoint plan) and GUIDED mode (your code issues commands directly), and what each implies for the pre-loaded flight plan.
- Why `MIS_RESTART = 0` on the autopilot is a critical pre-flight configuration, and what happens if it is not set correctly.
- Body-frame velocity commands: coordinate system, sign conventions, and how the blocking duration of `small_move_*` constrains the centering loop.
- Why `goto_point()` needs a timeout and how to handle the `False` return correctly in each context where it can occur.

3.5 Engineering Judgement and Quantitative Design

- How to derive pixel-level thresholds from physical requirements using camera geometry: given FOV, resolution, and altitude, how many pixels correspond to 1 meter of horizontal error?
- How to reason about a proportional velocity controller given a fixed 1-second block per correction: what is the maximum safe velocity that avoids overshoot?
- How to justify a safe delivery radius from first principles: package mass, descent velocity, tether length, and rotor wash clearance all factor in.
- When "continue with a logged warning" is the right response to a subsystem failure, and when it is not.

4. Mission Objective

The drone carries a first-aid care package suspended on a 3-meter (10-foot) tether cable with a servo-controlled release latch. The following sequence must occur *autonomously* and in order:

1. Take off and begin the pre-loaded search flight plan in AUTO mode.
2. Scan each camera frame with YOLOv8, looking for a person on the ground.
3. When a high-confidence detection occurs, record the drone's exact GPS position, altitude, and heading. Pause the flight plan.
4. Fly back to those coordinates. Restore the original heading. Wait for the drone to stabilize. Run YOLOv8 again on a fresh camera frame to confirm.
5. If confirmed: lock the histogram tracker on the target's bounding box. Issue proportional velocity corrections until the target is centered in the frame.
6. Estimate ground distance using camera geometry. Compute the GPS drop coordinate. Fly there.
7. Two-phase controlled descent: fast initially (package is safely high), then slow as the tether approaches full extension. Open the servo latch. Return home.

5. The Mission State Machine

The mission is governed by **five states**. The machine always begins in SEEK and always ends in RTL. The current state is logged at the front of every log entry and shown on the HUD.

5.1 State Diagram

```
+-------------------------------------------------------------------+
|                     MISSION STATE MACHINE                          |
|                                                                    |
|  +--------+  candidate    +---------+  confirmed   +------------+  |
|  |  SEEK  | ------------> | CONFIRM | -----------> |   TARGET   |  |
|  | (AUTO) | <------------ | (GUIDED)|              |  (GUIDED)  |  |
|  +--------+  not          +---------+              +-----+------+  |
|              confirmed                                   |         |
|                                                   centered on target |
|                                                          v         |
|                                                  +------------+    |
|                                                  |  DELIVER   |    |
|                                                  |  (GUIDED)  |    |
|                                                  +-----+------+    |
|                                                        |          |
|                                                 package released   |
|                                                        v          |
|                                                  +------------+    |
|                                                  |    RTL     |    |
|                                                  | (return)   |    |
|                                                  +------------+    |
+-------------------------------------------------------------------+
```

5.2 State Reference Table

State	Mode	Entry condition	What runs / How you exit
SEEK	AUTO	Mission start or failed CONFIRM	`check_for_initial_target()` every N frames (non-committing). Confidence > threshold → CONFIRM.
CONFIRM	GUIDED	YOLO fires in SEEK	Fly back → stabilize → detect again. Tracker confirms → TARGET; attempts exhausted → SEEK/AUTO.
TARGET	GUIDED	CONFIRM succeeds	`track_with_confirm()` + `adjust_to_target_center()` each frame. Centered → DELIVER; tracker lost entirely → SEEK.
DELIVER	GUIDED	TARGET centered	Geometry → navigate → descend → release. Package released → RTL.
RTL	RTL	DELIVER complete	Main loop exits; autopilot flies drone home.

5.3 Governing Design Rules

`conduct_mission()` owns all state transitions. Every `self.mission_mode =` assignment lives in this one method. No transitions are buried inside helper methods. A developer can read the loop top-to-bottom and understand the entire mission without leaving that function.

Helper methods do work and return results. `adjust_to_target_center()` returns `True` when centered and `False` while still adjusting. `deliver_package()` returns when done. The *transition decision* is always made by the caller.

CONFIRM and DELIVER are blocking. While they execute, the camera loop is suspended. This is intentional — each physical step must complete before the next begins.

6. Why the States Are Designed This Way

Why CONFIRM must physically fly back

By the time YOLOv8 fires and Python processes the result, the drone in AUTO mode may have travelled 20–50 m past the target. The detection came from a frame captured at the drone's *old* position. Confirming from the current position would fail because the target is now behind or outside the field of view. The drone must return to the lat/lon/alt of the original detection and restore its original heading — the camera's field of view is fixed to the drone body, so the wrong heading points the camera the wrong direction even at the correct GPS coordinates.

Why the histogram tracker does not initialize at first sight

`set_object_to_track()` builds a color signature from the target's bounding box crop. That signature needs a stationary drone, the target clearly visible in frame, and a confirmed, accurate bounding box. At first sight in SEEK the drone is cruising and has often already passed the target — none of these conditions hold. Initialising in CONFIRM, after the drone has returned and stabilized, gives the tracker ideal starting conditions.

Why HSV histogram tracking instead of CSRT

The previous version of this project used an OpenCV CSRT tracker. CSRT predicts each object's next position assuming the camera is approximately stationary. When the drone yaws or translates during a centering correction, every pixel in the frame shifts simultaneously — CSRT sees this as the target disappearing, and tracking fails exactly when the drone is actively trying to work. This is not a tuning problem; it is a fundamental mismatch between a static-camera tracker and a moving drone.

The histogram tracker matches targets by clothing color, not pixel position. A drone pan that shifts every bounding box by 30 pixels does not change a person's blue jacket. Histogram matching is inherently immune to camera ego-motion.

Why camera geometry instead of a rangefinder only

An Intel RealSense in depth mode provides the slant distance to the target. However, the depth sensor requires USB 3.0 bandwidth, returns noisy single-frame values, and adds a hardware dependency that can fail. Camera geometry uses the target's *pixel row* in the frame, the camera's vertical FOV, the IMU-measured tilt, and GPS altitude to estimate ground distance with no additional sensor. At mission altitudes, the geometric estimate is accurate to within 1–2 m — sufficient for the 3-meter delivery requirement. The configuration file lets you switch between both approaches with one setting.

Why two descent phases

Phase A descends quickly while the package is safely high. Phase B slows to a gentle rate as the 3-meter tether approaches full extension and the package nears the ground. A single fast descent would slam the payload and break the eggs. The altitude boundary between phases is a tunable parameter — it should be set based on your tether length and the specific payload mass.

7. Challenges You Will Face

Challenge: Confidence threshold selection

Your confidence threshold controls when a detection is worth pausing the flight plan for. Too high: the drone flies the entire search pattern and never commits to a sighting. Too low: every shadow triggers a 30-second CONFIRM cycle that wastes flight time and battery. The right value depends on your model, lighting, and terrain. You must reason about the false-positive / false-negative tradeoff quantitatively and justify your choice. What is the cost of a false positive in flight time? What is the cost of a false negative in mission success probability?

Challenge: Pixel-to-meter conversion

Your centering thresholds are in pixels, but the physical delivery requirement is in metres (within 10 feet of the target). You must derive the relationship between pixels and meters at your mission altitude using the camera's field of view and frame resolution. This derivation must be documented. "I tried different values until it seemed to work" is not an engineering answer.

Challenge: Proportional control and the 1-second block

Each `small_move_*` call blocks for 1 second. The drone physically moves for that second before the next camera frame is evaluated. This makes your effective correction rate approximately 1 Hz. At 0.5 m/s that is 0.5 m of movement per correction cycle. If your target acceptance radius is 0.5 m, a single overcorrection puts you outside the zone again. Think carefully about the relationship between velocity, block duration, and acceptance radius when choosing your parameters.

Challenge: goto_point convergence

`goto_point()` returns `False` if the drone has not arrived within `timeout_s` seconds. This can happen due to GPS oscillation near the target, wind opposing the approach, or a target point near the edge of the operational area. Your code must handle the `False` return at every call site. Think carefully about the correct response in each context — in the CONFIRM repositioning case the answer is different from the DELIVER approach case.

Challenge: Camera geometry distance estimation

`estimate_ground_distance_m()` depends on camera tilt, vertical FOV, target pixel row, and GPS altitude. An error in any one of these shifts the computed drop point. Know the formula. Know which parameter is most sensitive. Verify that your IMU tilt measurement produces a physically plausible angle (between 0 and 90 degrees, consistent with your mount) before the first flight.

Challenge: Tether physics and safe delivery radius

The package hangs on a 3-meter tether below the drone. During lateral corrections in TARGET mode, the tether swings. Aggressive corrections create pendulum oscillations that can cause early latch release or damage the payload. Slow, smooth corrections are an engineering requirement. When computing your delivery distance, reason about what "safe" means: close enough for an injured person to reach, far enough that the descending package and rotor wash do not injure them. Derive the radius; do not guess.

8. Files Provided to You

File	Status	Purpose
`pex03.py`	Skeleton	Your primary file. Contains `DroneMission` with `# TODO` sections you must complete.
`pex03_config.json`	Provided	Configuration file. Change parameters here — not in Python source code.
`mission_config.py`	Provided	Loads `pex03_config.json` into a typed `MissionConfig` object used throughout the mission.
`object_tracking_y8_histo.py`	Provided	YOLOv8 detection, HSV histogram identity tracking, camera stream interface. The only file `pex03.py` calls for perception.
`cam_handler.py`	Provided	RealSense camera pipeline with a background reader thread. Owns the pipeline; all other modules read from it.
`imu.py`	Provided	RealSense IMU reader and complementary filter. Used to measure camera tilt before arming.
`drone_lib.py`	Provided	DroneKit / MAVLink wrapper: arm, takeoff, mode switching, GPS navigation, body-frame velocity commands, descent.
`pex03_utils.py`	Provided	Geometry helpers: `estimate_ground_distance_m()`, `get_ground_distance()`, `calc_new_location()`, gripper servo control, frame logging.

9. Provided Module API Reference

9.1 object_tracking_y8_histo.py

```
# Start the RealSense camera pipeline.  Call ONCE at startup.
# Arguments come from MissionConfig --- do not hardcode values.
obj_track.start_camera_stream(
    use_distance=not config.use_estimated_distance,
    resolution_width=config.camera_resolution_width, ...)

# Load the YOLOv8 engine.  Call ONCE after camera starts.
obj_track.load_model(config.weights_path,
                     imgsz=config.model_imgsz,
                     conf=config.detect_confidence)

# Get the latest camera frame.  Returns None if unavailable --- always check.
frame = obj_track.get_cur_frame()
if frame is None:
    continue

# Non-committing YOLO scan (SEEK state).  Does NOT build tracker signature.
# Returns: (center, confidence, corner, radius, display_frame, bbox)
# confidence is None if no person was detected.
center, conf, corner, radius, frm, bbox = \
    obj_track.check_for_initial_target(
        frame, frm_display, show_img=self.virtual_mode)

# Lock the histogram tracker onto the confirmed target.
# Call ONLY after CONFIRM, once you have a confirmed bounding box.
```

```
# Builds the color signature used for identity matching.
obj_track.set_object_to_track(frame, bbox)

# Track + histogram match (TARGET state).  Returns same 6-tuple.
# confidence=None AND _target_track_id=None means miss budget exhausted.
center, conf, corner, radius, frm, bbox = \
    obj_track.track_with_confirm(frame, frm_display,
                                 show_img=self.virtual_mode)

# Useful module-level globals:
obj_track.FRAME_HORIZONTAL_CENTER    # int -- x centre of frame
obj_track.FRAME_VERTICAL_CENTER      # int -- y centre of frame
obj_track.FRAME_HEIGHT               # int -- frame height in pixels
obj_track._target_track_id           # None when tracker is not active
```

9.2 pex03_utils.py

```
# Camera-geometry ground distance (no rangefinder needed).
# Use config.effective_camera_tilt_deg -- always picks IMU-measured value
# when available, config fallback otherwise.
ground = pex03_utils.estimate_ground_distance_m(
    camera_tilt_deg  = config.effective_camera_tilt_deg,
    camera_fov_v_deg = config.camera_fov_v_deg,
    target_y_px      = float(center[1]),
    frame_height     = obj_track.FRAME_HEIGHT,
    altitude_m       = location.alt)
# Returns -1.0 if geometry is degenerate -- always check.

# Rangefinder path: Pythagorean ground distance.
ground = pex03_utils.get_ground_distance(altitude_m, slant_m)
# ground = sqrt(slant^2 - alt^2)

# Project a new GPS coordinate along a bearing.
drop_lat, drop_lon = pex03_utils.calc_new_location(
    lat, lon, heading_deg, distance_m)

# Open the servo latch (PWM 1087 on RC channel 7).
pex03_utils.release_grip(self.drone, seconds=2)

# Rangefinder average.  virtual_mode=True returns 35.0 m.
slant = pex03_utils.get_avg_distance_to_obj(
    5.0, self.drone, self.virtual_mode)
```

9.3 drone_lib.py

```
# Mode switching
drone_lib.change_device_mode(self.drone, "GUIDED")  # pause flight plan
drone_lib.change_device_mode(self.drone, "AUTO")    # resume flight plan
drone_lib.return_to_launch(self.drone)              # set RTL mode

# Navigation -- RETURNS True on arrival, False on timeout.
# Always capture the return value and respond appropriately.
arrived = drone_lib.goto_point(
    self.drone, lat, lon, speed_mps, alt_m, timeout_s=60)
if not arrived:
    log.warning("goto_point timed out -- proceeding from current position")
```

```
# Heading
drone_lib.condition_yaw(self.drone, heading_deg)   # 0 = North, clockwise

# Body-frame nudges.  EACH CALL BLOCKS FOR 1 SECOND.
# Drone moves approximately `velocity` meters before function returns.
drone_lib.small_move_forward(self.drone, velocity)
drone_lib.small_move_back(self.drone,    velocity)
drone_lib.small_move_left(self.drone,    velocity)
drone_lib.small_move_right(self.drone,   velocity)
drone_lib.small_move_down(self.drone,    velocity)  # positive = descend
```

Important

Every `small_move_*` call blocks for 1 second. The drone physically moves for that second before your code continues. At 0.5 m/s that is 0.5 m of movement before the next camera frame is evaluated. Choose your velocity values with this in mind — too fast and you overshoot your target; too slow and centering takes many minutes.

10. Student TODO Reference

Search `pex03.py` for `# TODO` to jump directly to each location. Every TODO includes a comment explaining what to implement and why.

Location	What to implement
`conduct_mission()` loop	`frame = obj_track.get_cur_frame()`
`conduct_mission()` GPS snapshot	`self.last_lat_pos`, `self.last_lon_pos`, `self.last_alt_pos`
SEEK state — `conf_level`	Mission-level confidence threshold. Derive and justify.
SEEK state — tracker init	`obj_track.set_object_to_track(frame, bbox)`
DELIVER → RTL transition	`drone_lib.return_to_launch(self.drone)`
`adjust_to_target_center()` — `pixel_forgiveness`	Derive from camera FOV and altitude. Document the calculation.
`adjust_to_target_center()` — `pixel_distance_threshold`	Threshold between fast and slow correction. Justify physically.
`adjust_to_target_center()` — velocity values	`yv` (fast and slow), `xv` (fast and slow).
`adjust_to_target_center()` — movement calls	Four `drone_lib.small_move_*` calls (fwd/back/left/right).
`deliver_package()` — rangefinder path	`pex03_utils.get_ground_distance(alt, slant_dist)`
`deliver_package()` — drop coordinate	`pex03_utils.calc_new_location(lat, lon, heading, dist)`
`deliver_package()` — fly to drop point	`drone_lib.goto_point(...)` including timeout handling.
`deliver_package()` — `alt_thresh`	Altitude boundary between fast and slow descent. Justify.
`deliver_package()` — descent commands	`drone_lib.small_move_down(...)` in both descent loops.
`deliver_package()` — release	`pex03_utils.release_grip(self.drone, seconds=2)`

11. Problem 1 — Detection Model (YOLOv8 VisDrone)

The provided model is **YOLOv8s** trained on the **VisDrone 2019** dataset — aerial video captured from drones at various altitudes, covering pedestrians, people, vehicles, and cyclists. YOLOv8s runs as a TensorRT-compiled engine for real-time inference on the Jetson. The model is configured for *detection only* — it does not use ByteTrack or any position-based tracker, because position-based tracking fails under camera ego-motion (see Section 11.2).

11.1 Known Model Limitations

1. **Orientation sensitivity.** Reliable on upright people; may miss people lying flat or at steep off-nadir angles. The target is a standing dressed mannequin, but viewing angle varies with search altitude and approach geometry.

2. **Scale sensitivity.** Trained on a range of altitudes. Flying significantly higher or lower than that range degrades detection probability. Your search altitude is a design decision with real

consequences for detection rate and confidence score distributions.

11.2 Two-Phase Perception Pipeline

1. **Detection (SEEK / CONFIRM):** `check_for_initial_target()` runs full YOLOv8 inference (≈30–50 ms) and returns the highest-confidence person bounding box. This is a *non-committing* scan — the tracker is not initialized.
2. **Histogram tracking (TARGET):** `track_with_confirm()` runs YOLOv8 *every frame*, computes an HSV color histogram for each detected person crop, and returns the detection whose histogram best matches the stored target signature. If no detection matches above the threshold for `TRACKER_MISSES_MAX` consecutive frames, the target is declared lost.

Note

Why histogram tracking instead of CSRT?
Position-based trackers (CSRT, Kalman, ByteTrack) predict the target's next *location* in the frame. When the drone yaws or translates to centre on the target, every pixel in the frame shifts simultaneously — the predictor sees this as the target disappearing. Tracking fails exactly when the drone is actively working.
HSV histogram matching identifies the target by *appearance* — clothing color. A pan that shifts every bounding box by 30 pixels does not change a person's blue jacket. The histogram is computed fresh from each detected bbox each frame, so there is no accumulated position error and no sensitivity to camera motion.

12. Problem 2 — Target Centering (TARGET Mode)

12.1 The Pixel Error Model

With the camera tilted downward, the frame center represents the point directly in the drone's forward view along the camera optical axis. The pixel error on each axis tells you which direction to command a correction:

$$dx = \text{target}_x - \text{frame_center}_x \qquad \begin{cases} dx > 0 & \text{target is to the RIGHT} \\ dx < 0 & \text{target is to the LEFT} \end{cases}$$

$$dy = \text{frame_center}_y - \text{target}_y \qquad \begin{cases} dy > 0 & \text{target is FORWARD} \\ dy < 0 & \text{target is BACKWARD} \end{cases}$$

12.2 Pixel-to-Meter Conversion (derive this for your altitude)

At mission altitude h with camera horizontal FOV θ_H over W pixels, and vertical FOV θ_V over H pixels:

$$\left.\frac{\text{m}}{\text{px}}\right|_{\text{horizontal}} = \frac{2h\tan(\theta_H/2)}{W} \qquad \left.\frac{\text{m}}{\text{px}}\right|_{\text{vertical}} = \frac{2h\tan(\theta_V/2)}{H}$$

Use these relationships to convert your physical delivery requirement (3 m radius) into a pixel acceptance zone, and to bound the maximum safe velocity given the 1-second blocking duration of each correction command.

12.3 Two-Speed Correction Strategy

Student TODO: Implement adjust_to_target_center()

Use a two-speed approach:

```
# Large error: fast correction to close the gap quickly.
# Small error: slow/fine correction to avoid overshooting the acceptance zone.
if abs(error) > pixel_distance_threshold:
    velocity = fast_velocity   # choose based on m/px at your altitude
else:
    velocity = slow_velocity   # fine tuning -- smaller than acceptance zone/2

# Y-axis corrections (forward/backward)
if self.direction_y == "F":
    drone_lib.small_move_forward(self.drone, velocity=yv)  # blocks 1 s
else:
    drone_lib.small_move_back(self.drone,    velocity=yv)

# X-axis corrections (left/right)
if self.direction_x == "R":
    drone_lib.small_move_right(self.drone, velocity=xv)
else:
    drone_lib.small_move_left(self.drone,  velocity=xv)
```

Hint

Each `small_move_*` call blocks for 1 second. At 0.4 m/s the drone moves ≈0.4 m before the next camera frame is evaluated. If your acceptance zone is 0.5 m across (15 pixels at 15 m altitude), a 0.4 m correction at the edge of the zone may just barely land inside — or overshoot. Start with conservative velocities and observe convergence in SITL before flying.

13. Problem 3 — Ground Distance Estimation

13.1 Camera Geometry Approach (no rangefinder)

When the camera is tilted α degrees below horizontal and the target appears at pixel row y in a frame of height H with vertical FOV θ_V, the depression angle to the target is:

$$\phi = \alpha + \left(\frac{y}{H} - 0.5\right)\theta_V$$

At drone altitude h above flat ground, the horizontal ground distance is:

$$d_{\text{ground}} = \frac{h}{\tan(\phi)}$$

This is implemented in `pex03_utils.estimate_ground_distance_m()`. Use `config.effective_camera_tilt_deg` as the tilt value — this automatically uses the IMU-measured angle when available, falling back to the configured value otherwise.

Important

If the depression angle ϕ is less than approximately 1° (target near the horizon), `tan(`ϕ`)` approaches zero and the distance estimate becomes meaninglessly large or negative. The function returns -1.0 in this case. **Always check the return value before computing the drop coordinate.**

13.2 Rangefinder Approach (if available)

If your hardware has a working rangefinder, set `use_estimated_distance: false` in `pex03_config.json`. The rangefinder path uses Pythagoras:

$$d_{\text{ground}} = \sqrt{d_{\text{slant}}^2 - h^2}$$

Average the rangefinder over 5 seconds to reduce noise:

```
slant = pex03_utils.get_avg_distance_to_obj(5.0, self.drone,
                                            self.virtual_mode)
if slant <= 0:
    # Invalid reading -- do not compute sqrt of a negative number.
    # Log a warning and abort delivery or use estimated distance as fallback.
    return
ground = pex03_utils.get_ground_distance(alt, slant)
```

13.3 Computing and Flying to the Drop Point

Student TODO: Compute drop coordinate and fly there

```
# Project (effective_drop_dist) meters along the drone's current heading.
# Fly short of the target by SAFE_DELIVERY_RADIUS to avoid injuring the person.
SAFE_DELIVERY_RADIUS = 3.0   # meters (derive and justify yours!)
effective_drop_dist = max(1.0, ground_dist - SAFE_DELIVERY_RADIUS)

new_lat, new_lon = pex03_utils.calc_new_location(
    lat, lon, heading, effective_drop_dist)

# goto_point returns True on arrival, False on timeout.
# If timeout: log a warning and deliver from current position.
arrived = drone_lib.goto_point(
    self.drone, new_lat, new_lon, speed=2.5, alt=alt, timeout_s=45)
if not arrived:
    log.warning("goto_point timed out -- delivering from current position.")
```

14. Problem 4 — Descent and Package Release (DELIVER Mode)

14.1 Two-Phase Descent Design

The descent has two phases separated by an altitude threshold:

- **Fast phase** (mission altitude → `alt_thresh`): The package is safely high. A faster velocity closes this gap efficiently.
- **Slow phase** (`alt_thresh` → 3.2 m): The 3-meter tether is approaching full extension. A slow, controlled rate ensures the package contacts the ground gently. 3.2 m is chosen to account for a typical 3-meter tether — the package touches down when the drone is just above that altitude.

Student TODO: Implement both descent loops and the release

```
# alt_thresh: altitude (m) where you switch from fast to slow descent.
# Derive from tether length.  Must be > 3.2 m.
alt_thresh = ???   # TODO: set and justify this value

# Phase 1: fast descent.
while self.drone.location.global_relative_frame.alt > alt_thresh:
    if self.drone.mode.name in ("RTL", "LAND"):
        return  # GCS abort -- stop immediately
    drone_lib.small_move_down(self.drone, velocity=???)  # TODO

# Phase 2: slow/fine descent.
while self.drone.location.global_relative_frame.alt > 3.20:
    if self.drone.mode.name in ("RTL", "LAND"):
        return
    drone_lib.small_move_down(self.drone, velocity=???)  # TODO

# Open servo latch (PWM 1087 on RC channel 7 for 2 seconds).
pex03_utils.release_grip(self.drone, seconds=2)
time.sleep(1)   # ensure latch has fully opened before climbing away
```

Hint

Choose your descent velocities with payload protection in mind. A 0.50 m/s fast phase closes 7 m (15 m altitude → 8 m threshold) in about 14 seconds. A 0.20 m/s slow phase covers the remaining 4.8 m in about 24 seconds. Budget roughly 40 seconds for the full descent sequence in your mission timeline.

15. Development Guide

Work through these stages in order. Do not attempt a live flight without completing every earlier stage.

15.1 Stage 1 — Camera and Detection (no drone)

Run `python object_tracking_y8_histo.py` standalone. Point the RealSense down at a person. Verify YOLOv8 detections appear, the tracker locks and shows a histogram match score, and FPS stays above 10. Use `in_debug=True` to detect cars instead of people for indoor lab testing.

15.2 Stage 2 — Configuration and Startup

Set `virtual_mode: true` in `pex03_config.json`. Run `python pex03.py`. Verify the config loads and the summary table is logged.

15.3 Stage 3 — SITL Connectivity

Start SITL, upload a waypoint search pattern, run the mission script. Verify the drone arms, takes off, enters AUTO, and logs `[SEEK] Scanning...` each iteration.

15.4 Stage 4 — Centering Loop (TARGET Mode)

Implement `adjust_to_target_center()`. Force `mission_mode = MISSION_MODE_TARGET` and a hardcoded off-centre `center` tuple. Verify the drone moves in the correct direction and converges.

15.5 Stage 5 — CONFIRM Sequence

Implement the `set_object_to_track()` call. Force a confidence above your threshold. Trace SEEK → CONFIRM → TARGET in the logs. Verify that `goto_point` is called, its return value is checked, heading is restored, and the tracker is initialized only after the second YOLO detection.

15.6 Stage 6 — Delivery Geometry (Unit Test First)

```
from student.pex03_utils import estimate_ground_distance_m, calc_new_location

# At 15m altitude, 37.5 degree tilt, target at frame center -> ~19.6m
dist = estimate_ground_distance_m(
    camera_tilt_deg=37.5, camera_fov_v_deg=58.0,
    target_y_px=240, frame_height=480, altitude_m=15.0)
print(f"Ground dist: {dist:.2f} m")   # expect ~19.6 m

lat, lon = calc_new_location(39.0, -104.0, 0, dist - 3.0)
print(f"Drop: {lat:.6f}, {lon:.6f}")
```

Then implement the full delivery sequence and verify altitude decrements in SITL.

15.7 Stage 7 — Full Integration (Real Camera, SITL)

Real RealSense + SITL. Run the IMU tilt measurement. Force a detection. Review the annotated PNG frames written to `mission_log_path`.

15.8 Stage 8 — Live Flight

Set `virtual_mode: false` in `pex03_config.json`. Conduct a bound tethered flight before the evaluated mission. Verify the gripper opens correctly.

16. SITL Testing Guide

```
sim_vehicle.py -v ArduCopter --console --map
```

Set `connect_string: "127.0.0.1:14550"` and `virtual_mode: true` in `pex03_config.json`.

	Can test in SITL
✓	All five state transitions (SEEK → CONFIRM → TARGET → DELIVER → RTL)
✓	Mode switching: AUTO ↔ GUIDED → RTL
✓	GPS navigation (`goto_point` including timeout handling)
✓	`condition_yaw` heading restoration
✓	Altitude-based descent loops
✓	Camera geometry distance calculation
✓	GPS drop coordinate projection
✓	Config loading, log output, on-disk frame writes
	Cannot test in SITL
×	YOLOv8 detection — test with `object_tracking_y8_histo.py` standalone
×	Depth sensor — `virtual_mode=true` returns a fixed 35.0 m
×	IMU tilt measurement — no camera connected in SITL
×	Gripper servo — bench test separately
×	Wind, vibration, tether swing, payload protection

17. Tunable Parameters

All parameters are set in `pex03_config.json`. Do not hardcode these values in `pex03.py`.

Parameter	**Config key**	**Default**	**Effect of increasing**
Detection confidence	`detect_confidence`	0.25	Fewer weak detections passed to state machine
Max confirm attempts	`max_confirm_attempts`	8	More retries before returning to SEEK
Acceptance radius	`target_acceptance_radius_px`	10 px	Longer centering required before delivery
Tracker miss budget	`tracker_misses_max`	20	More tolerant of brief target occlusion
Histogram threshold	`histo_match_threshold`	0.70	Stricter identity matching; reduces false swaps
Scan stride	`update_rate`	4	Scan every N frames during SEEK (CPU tradeoff)
Log rate	`image_log_rate`	4	Save every N frames to disk
Camera tilt (fallback)	`camera_tilt_deg`	37.5°	Affects all distance estimates (use IMU measurement)
Distance method	`use_estimated_distance`	true	`true` = camera geometry; `false` = rangefinder

18. Hardware Reference

Component	Details
Drone	Quadrotor with ArduPilot autopilot (Pixhawk or equivalent)
Companion CPU	NVIDIA Jetson; runs all Python code
Camera	Intel RealSense D455 — 640×480 BGR at 60 fps; built-in IMU (accelerometer: 200 Hz; gyroscope: 400 Hz)
Autopilot link	`/dev/ttyACM0` at 115200 baud
Gripper servo	RC channel 7; PWM 1087 = open, PWM 1940 = closed
Tether	3.0 m (≈10 ft) from drone latch to care package
Payload	First-aid box containing eggs

Pre-flight checklist:

1. `pex03_config.json` — `virtual_mode: false`, correct `connect_string`, correct model and log paths.
2. `python object_tracking_y8_histo.py` — camera streams, YOLOv8 detects, histogram tracker locks with a plausible match score.
3. Gripper bench test — latch opens cleanly on `release_grip()` command.
4. Mission uploaded: `drone.commands.count > 0`.
5. `MIS_RESTART = 0` on autopilot (mission resumes at current waypoint, not from the beginning).
6. IMU tilt measurement runs at startup and logs a value between 0 and 90 degrees.

19. Documentation Requirements

19.1 Help Documentation

Document all help received from sources other than your instructor or course-provided materials. Describe *what* was provided, *where* in the code it applied, and *who* provided it. If no outside help was received, write "NONE."

19.2 Project Log

Submit a project log alongside your code. It must include:

- The main technical challenges you encountered and how you resolved each one.
- A quantitative justification for every tunable parameter you set — confidence threshold, pixel thresholds, velocities, descent altitudes, delivery radius.
- Each team member's specific contributions to the project.
- A post-mission analysis: what worked, what did not, what you would change.

Important

A missing or vague project log results in a **12% deduction** from the final project score. The parameter justification section is mandatory — "we tried different values" is not acceptable documentation.

20. Evaluation and Grading

20.1 Competitive Ranking

Teams are ranked against each other on mission day. Scoring is competitive — your grade depends on your rank relative to the other teams.

Rank	Score	What earns this rank
1st	**100%**	Fastest total mission time *and* most accurate delivery. Both conditions must be met. A slow but accurate delivery does not automatically win over a fast delivery that is also accurate.
2nd	**95%**	Strong performance across the five scoring dimensions; second-best combined result.
3rd and below	**90%**	Mission completed successfully — target identified, package delivered within the 10-foot radius, eggs intact.
Failed to execute	**80% (starting point)**	Mission did not complete. Final score may be lower than 80% based on code review and team interviews (see below).

20.2 Scoring Dimensions

All teams that execute are evaluated on five dimensions. These determine rank among teams that all complete the mission:

1. **Target identification** — Was the correct person detected and confirmed without excessive false CONFIRM cycles?
2. **Delivery proximity** — How close did the package land to the target? The 10-foot (3-meter) requirement is the passing threshold; closer is better.
3. **Payload integrity** — Are the eggs unbroken after delivery?
4. **Mission speed** — Total elapsed time from takeoff to RTL confirmation. Faster is better, all else equal.
5. **Execution smoothness** — Excessive oscillation in TARGET mode, long pauses between state transitions, erratic descent, and aggressive corrections that visibly swing the tether are penalised here.

20.3 Teams That Fail to Execute

A team that fails to complete the majority of the mission receives a **starting score of 80%**. The final score will be determined by a combination of:

- **Code review.** Is the implementation logically correct? Are all `# TODO` items addressed? Are parameter values justified in the project log? Is the state machine correctly implemented?
- **Team interviews.** Can each team member explain their design decisions? Can they identify what went wrong and how they would fix it? Can they describe what their code was *trying* to do at the point of failure?

- **Partial credit.** States that functioned correctly before failure are credited. A team that successfully executed SEEK, CONFIRM, and TARGET before a delivery failure will score higher than a team that crashed on arming.

The score for a non-executing team may be lower than 80% if code review reveals fundamental logical errors, major TODOs left unaddressed, or team members who cannot explain their implementation.

A. Configuration File Reference (pex03_config.json)

The complete list of settings available in `pex03_config.json`, with the key names you need and brief descriptions. The full documentation with detailed explanations is inside the JSON file itself (in the `_comment_*` blocks).

Key	Default	Purpose
`connect_string`	`"127...14550"`	Autopilot address
`baud`	115200	Serial baud rate
`virtual_mode`	true	Sim vs. live hardware
`takeoff_altitude_m`	15.0	Takeoff altitude (metres)
`weights_path`	(see file)	Path to YOLOv8 engine
`mission_log_path`	(see file)	Directory for logs and PNG frames
`camera_name`	D455	Label for log messages
`camera_has_imu`	true	Whether camera has IMU
`camera_resolution_width`	640	Frame width (pixels)
`camera_resolution_height`	480	Frame height (pixels)
`camera_frame_rate`	60	Color stream FPS
`camera_fov_h_deg`	87.0	Horizontal FOV (degrees)
`camera_fov_v_deg`	58.0	Vertical FOV (degrees)
`camera_tilt_deg`	37.5	Fallback tilt if IMU unavailable
`camera_tilt_from_imu`	true	Measure tilt from IMU before arming
`camera_tilt_sample_duration_s`	2.0	IMU tilt sampling duration
`imu_enabled`	true	Master IMU switch
`imu_complementary_alpha`	0.98	Complementary filter gyro weight
`use_estimated_distance`	true	Camera geometry vs. rangefinder
`detect_confidence`	0.25	YOLOv8 minimum detection confidence
`model_imgsz`	640	YOLO inference resolution
`tracker_misses_max`	20	Frames before declaring target lost
`histo_match_threshold`	0.70	Minimum histogram correlation score
`max_confirm_attempts`	8	CONFIRM retry limit
`target_acceptance_radius_px`	10	Centering acceptance zone (pixels)
`update_rate`	1	SEEK scan stride (frames)
`image_log_rate`	4	Frame snapshot save stride